BTS CIEL · PROGRAMMATION C++ · EXERCICES

Exercices — Chapitre 6 : Représentation des données et opérations bit à bit

Cours associé : `C06_Representation_donnees_bit_a_bit` · **Fichiers de départ :** `02_Exercices/fichiers/E06/`

> **Méthode :** pour chaque exercice, faites **d'abord** le calcul à la main (binaire posé sur papier), **puis** vérifiez avec un programme. Le calcul à la main est attendu à l'évaluation.

N°	Titre	Niveau	Notions du cours
6.1	Conversions et opérations	1 — Application	§1, §4
6.2	Prévoir l'affichage	1 — Application	§1.3, §2, §4
6.3	XOR à la main	1 — Application	§6
6.4	Gestion de drapeaux	2 — Entraînement	§5
6.5	Décoder une trame textuelle	2 — Entraînement	§2
6.6	Valider une adresse IPv4	2 — Entraînement	§7, ch. 3
6.7	Construire des masques de préfixe	2 — Entraînement	§4.2, §7
6.8	Extraire les octets d'une adresse	2 — Entraînement	§5, §7
6.9	Démasquer une valeur	3 — Problème	§2, §6
6.10	Calculer une adresse réseau	3 — Problème	§7
6.11	Même sous-réseau ?	3 — Problème	§7
6.12	Clé décalée et aller-retour XOR	3 — Problème	§6
6.13	Registre d'état 16 bits	4 — Défi	§3 à §5

Le calculateur IPv4 complet (assemblage, masque, adresse réseau et diffusion) fait l'objet du **TP06**.

Niveau 1 — Application directe

Exercice 6.1 — Conversions et opérations

Objectif : passer d'une base à l'autre et appliquer les opérateurs bit à bit.

Énoncé :

1. Complétez le tableau :

Décimal	Binaire (8 bits)	Hexadécimal
10		
	0011 0000	
		0x7F
192		

2. Calculez à la main : `0b0101 & 0b0011`, `0b1100 | 0b1010`, `0b1111 ^ 0b0101`.
3. Calculez à la main : `5u << 1`, `5u << 2`, `20u >> 1`, `20u >> 2`. Quelle règle relie ces résultats aux multiplications et divisions par 2 ?
4. Vérifiez toutes vos réponses avec un programme C++.

Exercice 6.2 — Prévoir l'affichage

Objectif : relier extraction de quartets, XOR, affichage hexadécimal et codes ASCII.

Énoncé : sans exécuter le programme `ex6_2.cpp`, indiquez les trois lignes qu'il affiche, en détaillant chaque calcul en binaire. Vérifiez ensuite en compilant.

```cpp
#include <iostream>

int main() {
    unsigned int v{0xA5};

    std::cout << ((v >> 4) & 0xF) << ' ' << (v & 0xF) << '\n';
    std::cout << std::hex << (v ^ 0xFF) << std::dec << '\n';
    std::cout << static_cast<char>(0x41 + 2) << '\n';

    return 0;
}
```

Exercice 6.3 — XOR à la main

Objectif : appliquer le XOR avec une clé répétée.

Énoncé : on reprend l'exemple du cours (§6.2) : clé `10 20 30 40`, données masquées `53 69 75 0C 31` (en hexadécimal).

1. Démasquez à la main le **premier** octet : écrivez-le en binaire, appliquez le XOR avec l'octet de clé, convertissez en hexadécimal, puis en caractère.
2. Faites de même pour le **dernier** octet (indice 4). Quel octet de clé utilise-t-il ? Pourquoi ?

Résultat attendu : les caractères `C` et `!`, avec le détail des calculs.

Niveau 2 — Entraînement

Exercice 6.4 — Gestion de drapeaux

Objectif : tester, activer, désactiver des bits avec des masques.

Énoncé : l'état d'un équipement est codé sur 4 bits :

Bit	Signification
0	Alimentation
1	Capteur connecté
2	Alarme
3	Maintenance

Partant de l'état `0001`, effectuez dans l'ordre : activer le capteur, tester l'alarme (afficher le résultat), activer l'alarme, désactiver l'alimentation. Affichez l'état final en décimal.

Contraintes : une constante `constexpr` par bit ; calculez d'abord chaque état intermédiaire à la main.

Résultat attendu : les quatre états intermédiaires en binaire, le résultat du test d'alarme et l'état final en décimal.

Exercice 6.5 — Décoder une trame textuelle

Objectif : convertir des octets en caractères ASCII et isoler des champs.

Énoncé : un analyseur relève la trame `0x33 0x2F 0x34 0x31 0x2F 0x37`.

1. Convertissez chaque octet en décimal, puis en caractère ASCII.
2. La trame suit le format `adresse/commande/donnee` : donnez la valeur de chaque champ.
3. Écrivez le programme de décodage : il affiche la trame en texte, puis chaque champ.

Exercice 6.6 — Valider une adresse IPv4

Objectif : combiner des conditions pour valider les octets d'une adresse.

Énoncé :

1. Saisissez les quatre octets d'une adresse (quatre entiers) et vérifiez avec `&&` que chacun est compris entre 0 et 255. Affichez `Adresse valide` ou `Adresse invalide`.
2. Un laboratoire n'accepte que les adresses dont le dernier octet est compris entre 10 et 200 inclus : ajoutez ce contrôle, avec un message distinct.

Tests :

Saisie	Affichage attendu
`192 168 1 10`	`Adresse valide`, dernier octet accepte
`192 168 1 256`	`Adresse invalide`
`192 -1 1 10`	`Adresse invalide`
`192 168 1 9`	`Adresse valide`, dernier octet refuse
`192 168 1 200`	`Adresse valide`, dernier octet accepte

Exercice 6.7 — Construire des masques de préfixe

Objectif : construire un masque de sous-réseau par décalage.

Énoncé : construisez, sur un `std::uint32_t`, le masque des préfixes `/8`, `/16`, `/24` et `/26`, puis affichez-le en hexadécimal.

Contraintes : utilisez `0xFFFFFFFFu << (32 - prefixe)` ; le préfixe `/0` provoquerait un décalage de 32 bits (comportement indéfini) : traitez-le à part.

Résultat attendu :

```
/8  : FF000000
/16 : FFFF0000
/24 : FFFFFF00
/26 : FFFFFFC0
/0  : 0
```

Exercice 6.8 — Extraire les octets d'une adresse

Objectif : extraire des octets d'un entier de 32 bits.

Énoncé : l'adresse `192.168.1.10` est stockée dans un `std::uint32_t` initialisé à `0xC0A8010Au`. Extrayez chaque octet avec `>>` et `& 0xFF`, puis affichez l'adresse en notation pointée.

Résultat attendu :

```
192.168.1.10
```

Niveau 3 — Résolution de problèmes

Exercice 6.9 — Démasquer une valeur

Objectif : démasquer une trame XOR et interpréter les caractères obtenus.

Énoncé : une trame masquée contient quatre octets de données de la forme `T=xy`, où `xy` est une température.

```
Cle de masquage     : A1 B2 C3 D4
Donnees masquees    : F5 8F F1 E5
```

1. Démasquez à la main les 3e et 4e octets de données, en précisant les octets de clé utilisés, puis donnez la température.
2. Vérifiez que le premier octet correspond bien au caractère `T`.
3. Écrivez le programme de démasquage ; il affiche le texte démasqué, puis la température sous forme d'**entier** (calculée à partir des deux caractères chiffres).

Exercice 6.10 — Calculer une adresse réseau

Objectif : appliquer un masque par ET bit à bit.

Énoncé :

1. Pour `192.168.1.77/26` (masque `255.255.255.192`), convertissez 77 et 192 en binaire, calculez `77 & 192` et donnez l'adresse réseau. Vérifiez en C++.
2. Calculez l'adresse réseau de `192.168.1.230` avec `/24`, `/25`, `/26` et `/27`. Expliquez pourquoi elle change avec le préfixe.

Exercice 6.11 — Même sous-réseau ?

Objectif : décider si deux adresses appartiennent au même sous-réseau.

Énoncé : `192.168.10.25` et `192.168.10.200` sont-elles dans le même sous-réseau avec `/24` ? avec `/25` ? Justifiez en binaire, puis écrivez le programme de vérification : deux adresses sont dans le même sous-réseau si leurs adresses réseau sont égales.

Résultat attendu :

```
/24 : meme sous-reseau
/25 : sous-reseaux differents
```

Exercice 6.12 — Clé décalée et aller-retour XOR

Objectif : comprendre le rôle de l'indice de clé et vérifier la réversibilité du XOR.

Énoncé : on reprend le programme du cours qui démasque `CIEL!` (§6.2).

1. Remplacez `cle[i % cle.size()]` par `cle[(i + 1) % cle.size()]`. Avant d'exécuter, prévoyez le premier octet obtenu. Pourquoi le message devient-il illisible ?
2. Écrivez un programme qui **masque** le texte `CARNUS` avec la clé `10 20 30 40`, affiche les octets masqués en hexadécimal, puis les **démasque** et vérifie que le texte retrouvé est identique au texte de départ.

Résultat attendu (question 2) :

```
Masque   : 53 61 62 0E 45 73
Demasque : CARNUS
Aller-retour correct
```

Niveau 4 — Défi

Exercice 6.13 — Registre d'état 16 bits

Objectif : extraire et modifier des **champs** de plusieurs bits dans un registre.

Contexte : un module d'acquisition expose un registre d'état de 16 bits :

Bits	Champ	Signification
0 à 3	`code`	Code d'erreur (0 = aucune)
4 à 7	`tentatives`	Nombre de tentatives de connexion
8 à 11	`voie`	Numéro de la voie de mesure active
12 à 14	—	Réservés
15	`alarme`	1 = alarme active

Énoncé : le registre lu vaut `0x8A35`.

1. Écrivez sa valeur en binaire et délimitez les champs à la main.
2. Écrivez un programme qui extrait et affiche chaque champ (décalage puis masque).
3. Passez la voie active à 3 **sans modifier les autres bits** : effacez d'abord le champ, puis écrivez la nouvelle valeur. Affichez le nouveau registre en hexadécimal.

Contraintes :

- registre de type `std::uint16_t` ;

- position et masque de chaque champ en constantes `constexpr` ;
- les opérations sur un `std::uint16_t` sont calculées en `int` (promotion entière, §4.1) : reconvertissez le résultat avec `static_cast<std::uint16_t>(…)`.

Résultat attendu :

```
code       : 5
tentatives : 3
voie       : 10
alarme     : 1
Nouveau registre : 0x8335
```